DOCUMENT AI SYSTEM

Technical Architecture and Design

1. SYSTEM OVERVIEW

The Document AI system is an AI-powered document analysis and question answering application. Users upload PDF documents through a Streamlit interface. The backend processes documents, creates searchable representations, and allows users to ask natural language questions about uploaded documents. The application consists of a Streamlit frontend, FastAPI backend, PDF processing services, an embedding service, a vector database, and a large language model.

2. RAG PIPELINE

The system uses Retrieval-Augmented Generation, commonly called RAG, to answer questions about documents. During ingestion, the PDF is converted into text, divided into smaller chunks, embeddings are created, and embeddings are stored in a vector database. During question answering, the user's question is converted into an embedding. A similarity search retrieves the most relevant document chunks, which are then provided as context to the language model.

3. EMBEDDINGS

Embeddings are numerical representations of text that capture semantic meaning. Similar pieces of text tend to have similar vector representations. The system uses Sentence Transformers and the all-MiniLM-L6-v2 embedding model. Every document chunk and every user question is converted into an embedding before similarity search.

4. VECTOR DATABASE

The system uses ChromaDB as its vector database. ChromaDB stores document chunks together with vector embeddings and metadata such as filename, page number, and chunk identifier. Similarity search returns chunks closest to the question embedding.

5. LANGUAGE MODEL GENERATION

After relevant chunks are retrieved, they are combined into context for the language model. The application uses the Groq API to generate answers. The model receives the question and retrieved context and is instructed to answer using only that context. If the answer cannot be found, the system should say it could not find the answer in the document.

6. API ARCHITECTURE

The backend is implemented using FastAPI. The upload endpoint receives PDF files and processes them. The query endpoint receives a question and document filename and performs retrieval and answer generation. The documents endpoint returns documents stored in the vector database.

7. FRONTEND

The user interface is implemented using Streamlit. Users can upload PDF documents, select a document, ask questions, and view generated answers. Source information associated with retrieved chunks is also displayed.

8. DEPLOYMENT

The application is designed to be containerized using Docker. Backend and frontend services can run separately using Docker Compose. Persistent volumes can preserve uploaded documents and the ChromaDB database between container restarts.

9. SECURITY AND CONFIGURATION

Sensitive configuration such as the Groq API key is stored in environment variables rather than application source code. The .env file is used during local development and should never be committed to GitHub.

10. FUTURE IMPROVEMENTS

Future versions can include improved chunking, reranking models, relevance thresholds, authentication, document deletion, document-level access control, evaluation datasets, monitoring, and cloud deployment. Retrieval can also be improved by experimenting with different embedding models and retrieval strategies.